

STRUKDAT BAB 4

List of List (Multi Linked List)

1. Pengertian

List of List adalah struktur data yang terdiri dari:

- Linked List utama (parent)
- Linked List turunan (child)

Setiap parent dapat memiliki banyak child.

Contoh:

Jurusan Informatika

```
|  
├─ Andi  
├─ Budi  
└─ Citra
```

Jurusan Sistem Informasi

```
|  
├─ Deni  
└─ Eko
```

Representasi:

Parent 1

```
|  
+--> Child  
|  
+--> Child  
|  
+--> Child
```

Parent 2

```
|
+--> Child
|
+--> Child
```

2. Konsep Parent dan Child

Parent menyimpan data utama.

Child menyimpan data yang berelasi dengan parent.

Contoh:

Fakultas

```
|
+--> Mahasiswa
```

atau

Kategori Produk

```
|
+--> Produk
```

3. Struktur Child

```
struct Child{

    int id;

    struct Child* next;

};
```

4. Struktur Parent

```
struct Parent{  
  
    int id;  
  
    struct Child* firstChild;  
  
    struct Parent* next;  
  
};
```

5. Visualisasi Struktur

```
Parent A  
  |  
+--> Child 1  
  |  
+--> Child 2  
  |  
+--> Child 3
```

```
Parent B  
  |  
+--> Child 4  
  |  
+--> Child 5
```

6. Membuat Parent Baru

```
struct Parent* createParent(int id){
```

```
struct Parent* p =  
(struct Parent*)malloc(sizeof(struct Parent));  
  
p->id = id;  
  
p->next = NULL;  
  
p->firstChild = NULL;  
  
return p;  
  
}
```

7. Membuat Child Baru

```
struct Child* createChild(int id){  
  
    struct Child* c =  
    (struct Child*)malloc(sizeof(struct Child));  
  
    c->id = id;  
  
    c->next = NULL;  
  
    return c;  
  
}
```

8. Insert Parent

Contoh:

Parent A

Parent B

Parent C

Implementasi:

```
void insertParent(struct Parent** head,
                  struct Parent* p){

    p->next = *head;

    *head = p;

}
```

9. Insert Child

Contoh:

```
Informatika
|
+--> Andi
```

Tambah:

Budi

Hasil:

```
Informatika
|
+--> Andi
```

|
+--> Budi

Implementasi:

```
void insertChild(struct Parent* p,
                 struct Child* c){

    c->next = p->firstChild;

    p->firstChild = c;

}
```

10. Traversal Parent

```
struct Parent* curr = head;

while(curr != NULL){

    curr = curr->next;

}
```

11. Traversal Child

```
struct Child* temp =
curr->firstChild;

while(temp != NULL){
```

```
temp = temp->next;  
  
}
```

12. Traversal Seluruh Struktur

Informatika

```
|— Andi  
|— Budi
```

SI

```
|— Deni  
|— Eko
```

Implementasi:

Parent

↓

Seluruh Child

Parent Berikutnya

↓

Seluruh Child

13. Search Parent

```
while(curr != NULL){  
  
    if(curr->id == x){
```

```
        return curr;

    }

    curr = curr->next;

}
```

14. Search Child

Cari child pada parent tertentu.

```
while(temp != NULL){

    if(temp->id == x){

        return temp;

    }

    temp = temp->next;

}
```

15. Menghitung Jumlah Parent

```
int count = 0;
```

```
while(curr != NULL){
```



```
        count++;

        curr = curr->next;

    }

    _____
```

16. Menghitung Jumlah Child

```
int count = 0;
```

```
while(temp != NULL){

    count++;

    temp = temp->next;

}
```

17. Delete Child

Contoh:

Andi

Budi

Citra

hapus:

Budi

Hasil:
Andi
Citra

Pointer harus diperbarui agar node yang dihapus tidak lagi terhubung.

18. Delete Parent

Parent hanya boleh dihapus jika:

Tidak memiliki child

atau

Seluruh child ikut dihapus

19. Kompleksitas Operasi

Operasi	Kompleksitas
Insert Parent	$O(1)$
Insert Child	$O(1)$
Search Parent	$O(n)$
Search Child	$O(m)$
Traversal	$O(n+m)$
Delete Parent	$O(n)$
Delete Child	$O(m)$

20. Kesalahan Umum

Child Tidak Terhubung Ke Parent

Salah:

```
insertChild(NULL, child);
```

Memory Leak

Menghapus parent tetapi child tidak dibebaskan.

Parent Hilang

Pointer parent berubah tanpa disengaja.

Child Salah Parent

Child masuk ke parent yang tidak sesuai.

21. Implementasi Lengkap

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Child{

    int id;
    struct Child* next;

};

struct Parent{

    int id;

    struct Child* firstChild;

    struct Parent* next;

};

struct Parent* createParent(int id){

    struct Parent* p =
    (struct Parent*)malloc(sizeof(struct Parent));

    p->id = id;
    p->next = NULL;
    p->firstChild = NULL;

    return p;

}

struct Child* createChild(int id){

    struct Child* c =
    (struct Child*)malloc(sizeof(struct Child));

    c->id = id;
    c->next = NULL;
```

```
return c;
```

```
}
```
